



《人工智能综合设计》—网页解析与抽取



人工智能综合设计 教研组



提纲



人工智能综合设计01 网页解析与抽取

- □ 课程安排
- □ HTML文档
- □ 利用beautifulsoup抽取网页内容
- □ 附录：文档分析简介



课程安排

- Day 1 Html网页内容抽取和分词
- Day 2 爬虫设计和实现原理（爬取给定域名下的所有网页）
- Day 3 倒排索引结构
- Day 4 基于倒排索引的基本检索和复杂搜索
- Day 5 信息检索的评测
- Day 6 Web UI（实现简单的get、post结构）
- Day 7 latex的使用、系统调试
- Day 8 课程报告与展示



爬取网站

- 人大新闻网域名下的所有网页: <https://news.ruc.edu.cn/>





考核方式

- 提交代码和文档
- 做得好的同学可以申请报告：只有做报告的同学成绩才可能会是优秀



今日目标



- 给定html文档，完成python程序，从文档中提取所有链接；提取文档标题和正文，将标题和正文进行分词



提纲



人工智能综合设计01 网页解析与抽取

- □ 课程安排
- □ HTML文档
- □ 利用beautifulsoup抽取网页内容
- □ 附录：文档分析简介



浏览器是如何访问万维网的？

- 浏览器会向服务器发起一个**HTTP请求 (request)**
 - 请求行：GET /scienceandtechnology HTTP/1.1
 - 请求方法：GET 获取相应路径指定的资源
 - 资源路径： /scienceandtechnology
 - 协议版本：HTTP/1.1
 - 常见的请求方法：
 - **GET**：获取相应路径指定的资源
 - HEAD：请求资源但只返回响应头 (response header)
 - POST：向指定资源提交数据，常常用于上传表单
 - PUT：向指定资源上传最新内容
 - DELETE：请求服务器删除指定资源



浏览器是如何访问万维网的?

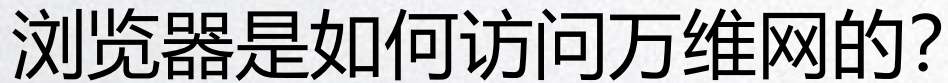
- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 响应头 (response header)

```
1 HTTP/1.1 200 OK
2 Date: Sat, 28 Nov 2020 04:54:45 GMT
3 Content-Type: text/html; charset=UTF-8
4 Link: <https://www.ruc.edu.cn/wp-json/>; rel="https://api.w.org/"
5 Vary: Accept-Encoding
6 Content-Encoding: gzip
7 Via: 3835
8 Cache-Control: max-age=600
```

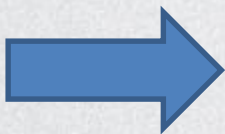


浏览器是如何访问万维网的？

- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 状态行： HTTP/1.1 200 OK
 - HTTP状态码：
 - 1xx消息——请求已被服务器接收，继续处理
 - 2xx成功——请求已成功被服务器接收、理解、并接受
 - 200 OK 请求的资源已返回
 - 3xx重定向——需要后续操作才能完成这一请求
 - 302 Found 常用于将GET请求重定向到其他路径
 - 4xx请求错误——请求含有词法错误或者无法被执行
 - 403 Forbidden 服务器拒绝执行请求
 - 404 Not Found 服务器找不到请求的资源
 - 5xx服务器错误——服务器在处理某个正确请求时发生错误
 - 500 Internal Sever Error 服务器出错



- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 响应内容类别： Content-Type: text/html; charset=UTF-8
 - 响应内容： UTF-8编码的**HTML文档**
 - 浏览器会将HTML文档渲染成可视化的网页展示在屏幕上

[illegible]

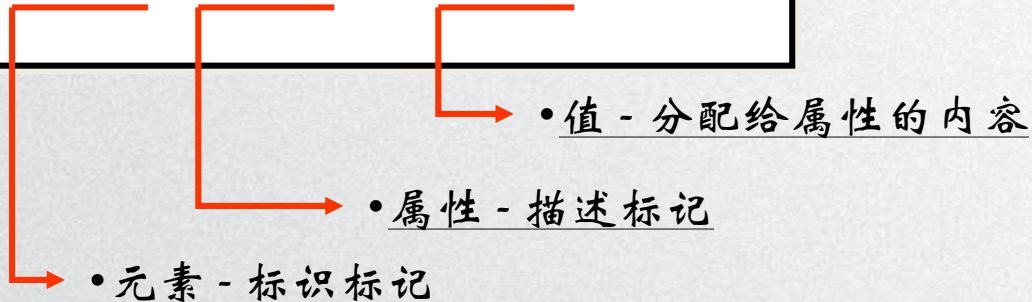
HTML 开发

- HTML 标记用于标记 HTML 文档的开始和结束

```
<HTML>  
.  
.  
.  
</HTML>
```

- HTML 标记的格式组成:

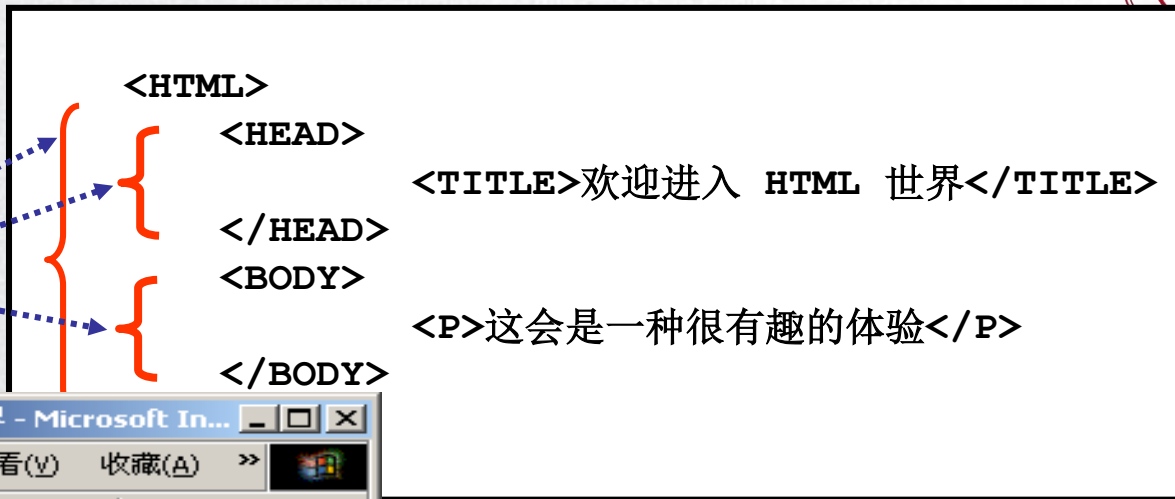
```
<ELEMENT ATTRIBUTE = value>
```



HTML 文档的结构

基本结构:

- HTML 部分
- 文档头部分
- 正文部分



运行结果



HTML头部信息(Head)

- HTML头部信息(head)里包含关于所在网页的信息。头部信息(head)里的内容，主要是被浏览器所用，不会显示在网页的正文内容里。
- 标题(title)
- 标题(title)是最常用的head信息。它不显示在HTML网页正文里，显示在浏览器窗口的标题栏里。
- 示例代码如下：
- `<html>`
- `<head> <title>HTML中文教程头部信息</title> </head>`
- `</html>`



使用META标记 2-1

1. 提供关于网页的信息

```
•<META name= "Author" content= "Graham Browne" >  
•<META name= "KEYWORDS" content= "... " >  
•<META name= "DESCRIPTION" content= "... ">
```

对网页的描述

根据关键词生成响应

获得文档的作者名称

2.应用： 关键词生成响应

```
<META http-equiv="Expires" content="Mon, 15 Sep 2003 14:25:27 GMT">
```

设置网页的到期值:响应

Expires: Mon, 15 Sep 2003 14:25:27 GMT

```
<META http-equiv="Content-Type" content="text/html; charset=gb2312">
```

设置网页使用gb2312:

页面显示中文



使用META标记 2-2

3.自动刷新页面

`<META http-equiv="Refresh" content="10; url=http://yourlink">`

应用：如网上实时新闻报道。

4.使用网页所使用的编码

`<META http-equiv="Content-Type" content="text/html; charset=gb2312">`

应用：如在不同浏览器上正确显示中文。



标题

- HTML用<h1>到<h6>这几个Tag来定义正文标题，从大到小。每个正文标题自成一段。



- `<html>`
- `<body>`
- `<h1>这是1号标题</h1>`
- `<h2>这是2号标题</h2>`
- `<h3>这是3号标题</h3>`
- `<h4>这是4号标题</h4>`
- `<h5>这是5号标题</h5>`
- `<h6>这是6号标题</h6>`
- `</body>`
- `</html>`



- 段落划分
- 在HTML里用<p>和</p>划分段落,<P>可以不成对出现。



段落标记



```
<HTML>
  <HEAD>
    <TITLE>欢迎使用 HTML</TITLE>
  </HEAD>
  <BODY BGCOLOR = lavender>
    <P>这会是一种很有趣的体验
    <P> 另一个段落元素
  </BODY>
</HTML>
```




使用对齐属性修饰标题

```
<HTML>
  <HEAD>
    <TITLE>学习HTML</TITLE>
  </HEAD>
  <BODY BGCOLOR=lavender>
    <P align = center>这会是一种很有趣的体验</P>
  </BODY>
</HTML>
```

Left: 左对齐
Center: 居中
Right: 右对齐

右对齐



使用对齐属性修饰标题

- `<html>`
- `<body>`
- `<h1 align="center">这是标题</h1>`
- `<p>上面的标题是居中显示的。</p>`
- `</body>`
- `</html>`



换行

- 通过使用
这个Tag, 可以在不新建段落的情况下换行。
没有Closing Tag。
- 用<p>换行是个坏习惯, 正确的是使用
,即在行末加入
。



换行标记



```
<HTML>
  <HEAD>
    <TITLE>欢迎使用 HTML</TITLE>
  </HEAD>
  <BODY BGCOLOR = lavender>
    <P>这会是一种很有趣的<BR>体验
    <P> 另一个段落元素
  </BODY>
</HTML>
```




HTML注释



- 在HTML文件里，你可以写代码注释，解释说明你的代码，这样有助于你和其他人日后能够更好地理解你的代码。
- 注释可以写在<!--和-->之间。浏览器是忽略注释的，你不会在HTML正文中看到你的注释。
- `<!-- This is a comment -->`



- `<html>`
- `<body>`
- `<!--这是代码注释-->`
- `<p>`代码注释是不会显示在网页里的。`</p>`
- `</body>`
- `</html>`



加入水平线

- 用<hr>这个Tag可以在HTML文件里加一条横线
- <Hr>没有结束标签



- `<html>`
- `<body>`
- `<p>暑期编程实践</p>`
- `<hr>`
- `<p>这是一段8天的编程生活</p>`
- `</body>`
- `</html>`



HTML利用超链接打开链接文件

- HTML用<a> 来表示超链接
- <a> 可以指向任何一个文件源：一个HTML网页，一个图片，一个影视文件等。用法如下：
- 链接的显示文字
- 点击<a> 当中的内容，即可打开一个链接文件，href属性则表示这个链接文件的路径。



- `<html>`
- `<body>`
- `<a href=" title.html">这是一个链接</a><br>`
- `<a href= "http://www.ruc.edu.cn" >中国人民大学主页链接</a>`
- `</body>`
- `</html>`



- 链接的target属性
- 使用链接的target属性，可以在一个新窗口里打开链接文件。



- `<html>`
- `<body>`
- `<a href=" 网页测试2.htm">这是一个链接</a> <br>`
- `中国人民大
学主页链接`
- `</body>`
- `</html>`



- 链接的title属性
- 使用 链接的title 属性，可以让鼠标悬停在超链接上的时候，显示该超链接的文字注释。



- `<html>`
- `<body>`
- `<a href=" 网页测试2.htm">这是一个链接</a> <br>`
- `<a href= "http://www.ruc.edu.cn" target= blank title= "中`
国人民大学主页链接">中国人民大学主页链接
- `</body>`
- `</html>`



- 链接到email地址
- 在网站中，你经常会看到“联系我们”的链接，一点击这个链接，就会触发你的邮件客户端，比如Outlook Express，然后显示一个新建mail的窗口。用<a>可以实现这样的功能。



- `<html>`
- `<body>`
- `<a href=" 网页测试2.html">这是一个链接</a> <br>`
- `<a href= "http://www.ruc.edu.cn" target= blank title= "中国人民大学主页链接">新浪站点链接</a>`
- `<a href = "mailto:web@ruc.edu.cn" >联系中国人民大学</a>`
- `</body>`
- `</html>`



HTML相对路径(Relative Path)和绝对路径(Absolute Path)

- HTML有2种路径的写法：相对路径和绝对路径。
- HTML相对路径(Relative Path)
 - 同一个目录的文件引用
 - 如果源文件和引用文件在同一个目录里，直接写引用文件名即可。
 - `<a href=" 网页测试2.html">这是一个链接</a>`
- 如何表示上级目录
 - `../`表示源文件所在目录的上一级目录，`../../`表示源文件所在目录的上上级目录，以此类推。
- 如何表示下级目录
 - 引用下级目录的文件，直接写下级目录文件的路径即可。



HTML绝对路径(Absolute Path)

- HTML绝对路径(absolute path)指带域名的文件的完整路径。
- `<a href= " http://www.ruc.edu.cn/index.html" target=_blank >`
新浪站点链接`</a>`



HTML图片(Images)

- 用 `<img>` 这个 Tag 可以在HTML里面插入图片。最基本的语法如下：
- `` url表示图片的路径和文件名。比如 url 可以是 [images/logo/blabla_logo01.gif](#)，也可以是个相对路径 `"../images/logo/blabla_logo01.gif"`。



- `<html>`
- `<body>`
- `<p>`
- 图片: ``
- `</p>`
- `</body>`
- `</html>`

- **图片align属性**
- 用align属性，可以改变水平对齐方式(居左、居中、居右)。
- 用valign属性，可以改变图片的垂直(居上、居中、居下)对齐方式
- **图片的大小**
- 在缺省状况下，图片显示原有的大小。你可以用height和width属性改变图片的大小。不过图片的大小一旦被改变，图片会相应放大或缩小，显示出来的结果可能会很难看。

- `<html>`
- `<body>`
- `<img src= "ruc.jpg" width=150 heigh=50 >`
- `</body>`
- `</html>`



HTML背景颜色和背景图片

- HTML的 `<body>` 有两个关于背景的属性,
- **bgcolor属性**
- bgcolor属性用来设置HTML网页的背景颜色。
- **background属性**
- background属性用来设置HTML网页的背景图片。
background属性值就是背景图片的路径和文件名,
如果背景图片小于网页显示窗口, 那么这个背景图
片会自动重复。



- `<html>`
- `<body bgcolor= "green">`
- `<h2>看，这个页面是绿色的。</h2>`
- `</body>`
- `</html>`

- `<html>`
- `<body background= "ruc.jpg">`
- `<h3>这个网页有背景图片哦! </h3>`
- `<p>如果图片比页面小, 图片会自动重复. </p>`
- `</body>`
- `</html>`



如何创建HTML表格

- HTML表格用<table>表示。
- 一个表格可以分成很多行，用<tr>表示；
- 每行又可以分成很多单元格，用<td>表示。
- 这三个Tag是创建表格最常用的Tag。

- `<html>`
- `<body> <table width="150" border="1">`
- `<tr>`
- `<td>1</td> <td>2</td> <td>3</td>`
- `</tr>`
- `<tr>`
- `<td>4</td> <td>5</td> <td>6</td>`
- `</tr>`
- `<tr>`
- `<td>7</td> <td>8</td> <td>9</td>`
- `</tr>`
- `</table>`
- `</body>`
- `</html>`



HTML常用文本格式

- HTML定义了一些文本格式的Tag，比如利用Tag，可以将字体变成粗体或者斜体。从下面的示例，可以了解各种文本格式Tag如何改变HTML文本的显示:
- 常用文本格式Tag
- `<b> </b>` 粗体bold
- `<i> </i>` 斜体
- `<U> </U>` 下划线



HTML字体(Fonts)

- 在HTML里，可以用font这个元素及其属性来设定字体的大小，颜色和字体风格。
- **字体大小**
- 用字体大小属性(size)来设定字体的大小。示例代码如下：
- `<p> 这一段的字体大小的值是2。
 </p>`
- **字体风格**
- 用face属性来设定字体风格。示例代码如下：
- `<p> 这一段的字体风格是arial。
 </p>`
- **字体颜色**
- 用颜色属性(color)来设定字体颜色。

- `<html>`
- `<head> <title> 字体大小 font size </title> </head>`
- `<body>`
- `<p> 这段文字的字体大小值为1。
 </p>`
- `<p> 这段文字的字体大小值为2。
 </p>`
- `<p> 这段文字的字体大小值为3。
 </p>`
- `<p> 这段文字的字体大小值为4。
 </p>`
- `</body>`
- `</html>`



- `<html>`
- `<head> <title>字体风格 font face</title> </head>`
- `<body>`
- `<p>`以下第一段用的是arial字体，第二段用的是courier字体，第三段用的是verdana字体。`</p>`
- `<p> arial courier
verdana </p>`
- `<p> arial courier
verdana </p>`
- `<p> arial courier
verdana </p>`
- `</body>`
- `</html>`



- `<html>`
- `<head> <title>字体颜色 font color</title> </head>`
- `<body>`
- `<p>`这段文字用的是缺省的字体颜色。`</p>`
- `<p> <font color="#FF0000">`这段文字的字体颜色为红色。`</font> </p>`
- `</body>`
- `</html>`



HTML表单(Forms)

- HTML表单(Form)是HTML的一个重要部分，主要用于采集和提交用户输入的信息。
- 举个简单的例子，一个让用户输入姓名的HTML表单(Form)。示例代码如下：

- `<html>`
- `<head> <title>输入用户姓名</title> </head>`
- `<body>`
- `<form action= "yonghu.asp" method="get">`
- 请输入你的姓名:
- `<input type="text" name="yourname">`
- `<input type="submit" value="提交">`
- `</form>`
- `</body>`
- `</html>`



提纲



人工智能综合设计01 网页解析与抽取

- □ 课程安排
- □ HTML文档
- □ 利用beautifulsoup抽取网页内容
- □ 附录：文档分析简介



使用Beautifulsoup库解析HTML文档

- BeautifulSoup是一个用于处理和解析HTML或XML文档，以从中提取信息的Python库。
 - 根据HTML和XML语法建立**解析树**，进而高效的解析其中的内容
 - 具有一定的错误容忍能力：能从不完整、有错误的HTML文档中提取信息
 - 安装方法：
 - pip install beautifulsoup4 (注意不是BeautifulSoup)
 - pip install lxml (高效的解析器，可选)
 - pip install html5lib (专门解析HTML的解析器，可选)
 - 主页：<https://www.crummy.com/software/BeautifulSoup/>
 - 文档：<https://www.crummy.com/software/BeautifulSoup/bs4/doc>
<https://www.crummy.com/software/BeautifulSoup/bs4/doc.zh/>



使用Beautifulsoup库解析HTML文档

- BeautifulSoup库采用面向对象思想实现：
 - 首先使用HTML文档创建一个BeautifulSoup对象

```
[25]: from bs4 import BeautifulSoup
      soup = BeautifulSoup(html_doc, 'html.parser') # 使用自带的解析器, 解析上述html文档
      # soup = BeautifulSoup(html_doc, 'html5lib') # 也可以使用html5lib中的解析器

      print(soup.prettify()) #将解析后的html文档按照标准的缩进结构输出
```



使用Beautifulsoup库解析HTML文档

- Beautifulsoup库采用面向对象思想实现：
 - 可以使用<a>.的方式调用对象属性

```
[30]: soup.title
[30]: <title>中国人民大学 | RENMIN UNIVERSITY of CHINA</title>

[31]: soup.title.name
[31]: 'title'

[32]: soup.title.string
[32]: '中国人民大学 | RENMIN UNIVERSITY of CHINA'

[33]: soup.p # 第一个p元素
[33]: <p class="title"><b>理工学部</b></p>

[38]: soup.a # 第一个a元素
[38]: <a class="school" href="http://envi.ruc.edu.cn" id="link1">环境学院</a>
```



使用Beautifulsoup库解析HTML文档

- BeautifulSoup库采用面向对象思想实现：
 - 可以使用[<a>.\(\)](#)的方式调用对象方法（调用函数）

```
[39]: soup.find_all('a') # 找到所有a元素

[39]: [<a class="school" href="http://envi.ruc.edu.cn" id="link1">环境学院</a>,
      <a class="school" href="http://info.ruc.edu.cn" id="link2">信息学院</a>,
      <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>,
      <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>,
      <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>,
      <a class="school" href="http://math.ruc.edu.cn/" id="link6">数学学院</a>,
      <a class="school" href="http://ai.ruc.edu.cn/" id="link7">高瓴人工智能学院</a>]

[41]: soup.p['class'] # 访问元素属性

[41]: ['title']

[42]: soup.find(id="link7") # 按照属性查找元素, find返回第一个结果

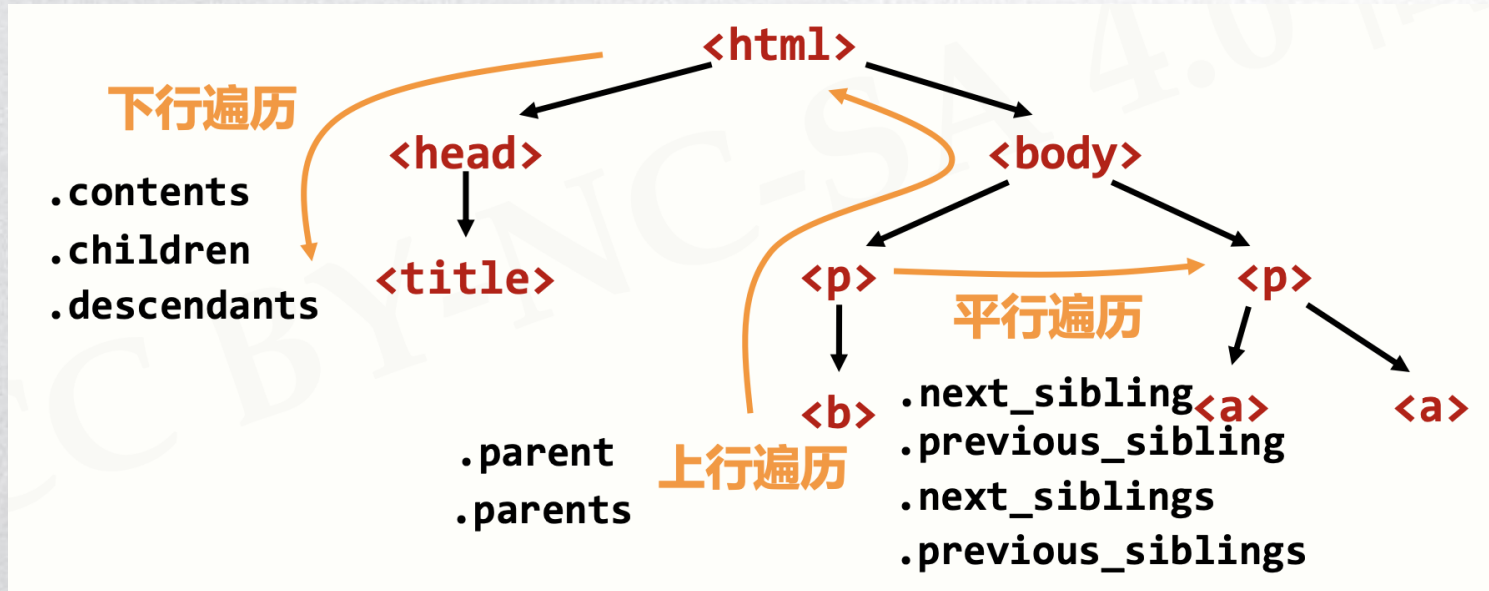
[42]: <a class="school" href="http://ai.ruc.edu.cn/" id="link7">高瓴人工智能学院</a>

[43]: soup.find(class_="school") # 按照属性查找元素, 因为class是python关键字, 所以用class_代替

[43]: <a class="school" href="http://envi.ruc.edu.cn" id="link1">环境学院</a>
```

使用Beautifulsoup库解析HTML文档

- Beautifulsoup会建立HTML文档的**解析树**
 - 树中的每个节点均为一个对象
 - 可按照解析树的结构来访问和遍历需要提取的信息





使用Beautifulsoup库解析HTML文档

- BeautifulSoup会建立HTML文档的解析树
 - 树中的每个节点均为一个对象

```
[49]: # 树中每一个节点都是一个对象, 对象属于不同的类
      print(type(soup)) # 整个HTML文档是一个BeautifulSoup对象

      print(type(soup.title)) # 每个元素是一个Tag对象
      print(type(soup.p))

      print(type(soup.title.string)) # 元素的内容是一个NavigableString对象 (类似String)

<class 'bs4.BeautifulSoup'>
<class 'bs4.element.Tag'>
<class 'bs4.element.Tag'>
<class 'bs4.element.NavigableString'>
```



使用Beautifulsoup库解析HTML文档

- Tag对象:

```
[66]: # Tag对象是最常见的对象, 下面列举一些它具有的属性和方法  
tag = soup.find('a') # 用第一个a元素作为例子
```

```
tag.name # 元素的名称
```

```
[66]: 'a'
```

```
[67]: tag.attrs # 元素的所有属性
```

```
[67]: {'href': 'http://envi.ruc.edu.cn', 'class': ['school'], 'id': 'link1'}
```

```
[68]: tag['href'] # 访问元素的属性
```

```
[68]: 'http://envi.ruc.edu.cn'
```

```
[69]: tag.string # 访问元素的内容 (尖括号中的字符串)
```

```
[69]: '环境学院'
```



使用Beautifulsoup库解析HTML文档

- 在解析树上遍历和查找元素：

```
[56]: # 在解析树上遍历和查找元素
```

```
tag = soup.find('p', class_='school') # 找到理学院对应的<p>元素
tag
```

```
[56]: <p class="school">
```

```
    理学院: <br/>
```

```
    <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a><br/>
```

```
    <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a><br/>
```

```
    <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>
```

```
</p>
```

```
[57]: tag.contents # 该元素下所有的所有内容
```

```
[57]: ['\n          理学院: ',
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>,
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>,
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>,
```

```
       '\n']
```

```
[64]: tag.contents # 以list形式获得该元素下所有的所有内容
```

```
[64]: ['\n          理学院: ',
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>,
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>,
```

```
       <br/>,
```

```
       '\n',
```

```
       <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>,
```

```
       '\n']
```



使用Beautifulsoup库解析HTML文档

- 在解析树上遍历和查找元素：
 - .children

```
[60]: for child in tag.children: #遍历该元素下所有子元素
      print(type(child), child)

<class 'bs4.element.NavigableString'>
      理学院:
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>
<class 'bs4.element.NavigableString'>
```




使用Beautifulsoup库解析HTML文档

- 在解析树上遍历和查找元素：
 - `.descendants`

```
[62]: for child in tag.descendants: # 递归的遍历该元素下所有子元素
      print(type(child), child)

<class 'bs4.element.NavigableString'>
      理学院:
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>
<class 'bs4.element.NavigableString'> 物理学系
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>
<class 'bs4.element.NavigableString'> 化学系
<class 'bs4.element.Tag'> <br/>
<class 'bs4.element.NavigableString'>

<class 'bs4.element.Tag'> <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>
<class 'bs4.element.NavigableString'> 心理学系
<class 'bs4.element.NavigableString'>
```



使用Beautifulsoup库解析HTML文档

- 查找元素的父节点

```
[74]: tag.parent # 元素的父节点
```

```
[74]: <p class="content">
      <a class="school" href="http://envi.ruc.edu.cn" id="link1">环境学院</a><br/>
      <a class="school" href="http://info.ruc.edu.cn" id="link2">信息学院</a>
      <p class="school">
          理学院: <br/>
          <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a><br/>
          <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a><br/>
          <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>
      </p>
      <a class="school" href="http://math.ruc.edu.cn/" id="link6">数学学院</a><br/>
      <a class="school" href="http://ai.ruc.edu.cn/" id="link7">高瓴人工智能学院</a><br/>
      </p>
```

```
[76]: for parent in tag.parents: # 向上遍历元素的所有父节点
      print(parent.name)
```

```
p
body
html
[document]
```



使用Beautifulsoup库解析HTML文档

- 访问兄弟节点：
 - 部分兄弟节点是空行

```
[80]: # 兄弟节点
print(tag.next_sibling)
for sibling in tag.next_siblings:
    print(sibling)
```

```
<a class="school" href="http://math.ruc.edu.cn/" id="link6">数学学院</a>
<br/>
```

```
<a class="school" href="http://ai.ruc.edu.cn/" id="link7">高瓴人工智能学院</a>
<br/>
```

```
[81]: print(tag.previous_sibling)
for sibling in tag.previous_siblings:
    print(sibling)
```

```
<a class="school" href="http://info.ruc.edu.cn" id="link2">信息学院</a>
```

```
<br/>
```

```
<a class="school" href="http://envi.ruc.edu.cn" id="link1">环境学院</a>
```



使用Beautifulsoup库解析HTML文档

- 在中间节点上同样可以使用find()和find_all()方法：
 - 搜索范围会限制在所有直接和间接子节点上

```
[82]: # 在中间节点上同样可以使用find()和find_all()方法
      tag.find('a')
```

```
[82]: <a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>
```

```
[83]: tag.find_all('a')
```

```
[83]: [<a href="http://www.phys.ruc.edu.cn/" id="link3">物理学系</a>,
      <a href="http://chem.ruc.edu.cn/" id="link4">化学系</a>,
      <a href="http://psy.ruc.edu.cn/" id="link5">心理学系</a>]
```




抓取html文档中的所有超链接

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')
```

```
all_links = set()
for anchor in soup.find_all('a'):
    href = anchor.attrs.get("href")
    if (href != "" and href != None):
        all_links.add(href)
print(all_links)
```



不规范的URL

- 相对路径
 - jtgk/jtjj/index.htm
- URL中多余的字符/默认后缀
 - <http://hqjt.ruc.edu.cn/#>
 - zjhqr/index.htm (http://hqjt.ruc.edu.cn/index.htm)
- 缺少字符串等其它不规范
 - 缺少"/": "http://www.cnblogs.com/shuchao"
- 不需要爬取的链接
 - images/2018-12/41f44792caaa40068cb23ff156aec557.png



URL normalization

- URL规范化
- <https://pypi.org/project/url-normalize/>
- 安装: `pip install url-normalize`

```
from url_normalize import url_normalize  
print(url_normalize('http://hqjt.ruc.edu.cn/#'))
```

```
http://hqjt.ruc.edu.cn/
```



相对路径

- cur_url: 当前html文件的URL
- href: html文件中的一个超链接

```
cur_url = 'http://hqjt.ruc.edu.cn/'  
href = 'jtgk/jtjj/index.htm'  
if not href.startswith('http'):  
    href = url_normalize(cur_url + href)  
print(href)
```

<http://hqjt.ruc.edu.cn/jtgk/jtjj/index.htm>



抓取标题并分词

```
# html title words
import jieba
soup2 = BeautifulSoup(html_doc2, 'html.parser')
title_words = jieba.lcut(soup2.title.string)
print(title_words)

# multilevel titles
doctitle_words = []
for t in soup2.find_all({"h1", "h2", "h3"}):
    if t.string is not None:
        temp_words = jieba.lcut(t.string)
        doctitle_words += temp_words
print(doctitle_words)
```

抓取标题并分词

- 英文单词间有空格：split()分词
- jieba是Python中一个重要的第三方中文分词函数库

import jieba

- jieba库是第三方库，不是安装包自带，需要通过pip指令安装

pip install jieba # pip3 install jieba

- 解析

函数	描述
<code>jieba.cut(s)</code>	精确模式，返回一个可迭代的数据类型
<code>jieba.cut(s, cut_all=True)</code>	全模式，输出文本s中所有可能单词
<code>jieba.cut_for_search(s)</code>	搜索引擎模式，适合搜索引擎建立索引的分词结果
<code>jieba.lcut(s)</code>	精确模式，返回一个列表类型，建议使用
<code>jieba.lcut(s, cut_all=True)</code>	全模式，返回一个列表类型，建议使用
<code>jieba.lcut_for_search(s)</code>	搜索引擎模式，返回一个列表类型，建议使用
<code>jieba.add_word(w)</code>	向分词词典中增加新词w



Jieba库

- 用法示例

```
In [73]: import jieba  
jieba.lcut("中华人民共和国是一个伟大的国家")
```

```
Out[73]: ['中华人民共和国', '是', '一个', '伟大', '的', '国家']
```

```
In [71]: jieba.lcut("中华人民共和国是一个伟大的国家", cut_all=True)
```

```
Out[71]: ['中华',  
          '中华人民',  
          '中华人民共和国',  
          '华人',  
          '人民',  
          '人民共和国',  
          '共和',  
          '共和国',  
          '国是',  
          '一个',  
          '伟大',  
          '的',  
          '国家']
```

```
In [72]: jieba.lcut_for_search("中华人民共和国是一个伟大的国家")
```

```
Out[72]: ['中华', '华人', '人民', '共和', '共和国', '中华人民共和国', '是', '一个', '伟大', '的', '国家']
```



抓取正文并分词

```
# html body words
# 每段分别分词，再合并
body_words = []
for p in soup2.find_all('p'):
    if p.string is not None:
        temp_words = jieba.lcut(p.string)
        body_words += temp_words
print(body_words)
#find_all(text=True)

# 先合并所有正文，再分词
parsed_text = "".join(
    [p.string for p in soup2.find_all('p') if p.string is not None]
)
if parsed_text is not None:
    body_words = jieba.lcut(parsed_text)
else:
    body_words = []
print(parsed_text)
print(body_words)
```




谢谢！



不规范的URL

1 URL中多余的字符

- 1.1 子域名的URL中包含"www": "http://www.shuchao.cnblogs.com/"
- 1.2 含有默认端口: "http://www.cnblogs.com:80/shuchao/"
- 1.3 松散的URL: "http://www.chapters.indigo.ca/books/amazon-sucks-donkey-balls/9780470170779-item.html"
- 1.4 多余默认文件名index.html,default.aspx等:
"http://www.cnblogs.com/shuchao/index.html"
- 1.5 文件路径中
 - (1) 多余的"/": "http://www.cnblogs.com/shuchao//"
 - (2) 多余的点修饰串: "x/y/z/http://www.cnblogs.com/a/b/http://www.cnblogs.com/./page.html"
- 1.6 查询串中多余的
 - (1) ? (空查询串): http://www.cnblogs.com/shuchao?
 - (2) &
 - (3) 无用的查询变量: http://www.example.com/display?id=123&fake=fake



不规范的URL

2 URL缺少字符串

- 2.1 缺少"/":"http://www.cnblogs.com/shuchao"
- 2.2 查询串缺少名称或者值:"http://www.example.com/display?id=" 或者 "http://www.example.com/display?=123"

3 其他不规范的URL

- 3.1 "http://shuchao.cnblogs.com/" 与 "http://www.cnblogs.com/shuchao/" 其实是相同的内容
- 3.2 使用IP代替域名
- 3.3 含有扩充字符(extended characters), 大小写敏感 ("http://www.google.cn/Intl/zh-CN/about.html" 和 "http://www.google.cn/intl/zh-CN/about.html")
- 3.4 "+"和"%20"混用
- 3.5 查询变量顺序混乱:"http://www.example.com/test.aspx?bar=1&a=test"
- 3.6 含临时的状态变量:http://www.example.com/test?back=/prevpage.aspx



URL normalization

- 1. URL协议名和主机名小写化
HTTP://WWW.EXAMPLE.com/test ->
http://www.example.com/test
- 2. escape序列转化为大写, 因为escape序列大小敏感
%3a -> %3A
- 3. 删除Fragment(#)
http://www.example.com/test/index.html#seo ->
http://www.example.com/test/index.html
- 4. 删除空查询串的'?'
http://www.example.com/test? -> http://www.example.com/test
- 5. 删除默认后缀
http://www.example.com/test/index.html -
> http://www.example.com/test/
- 6. 删除多余的点修复符
http://www.example.com/./a/b/./c/./d.html ->
http://www.example.com/a/c/d.html



URL normalization

- 7.删除多余的"www"

http://www.test.example.com/ -> http://test.example.com/

- 8.对查询变量排序

http://www.example.com/test?id=123&fakefoo=fakebar →
http://www.example.com/test?id=123 \

- 9.删除取默认值的变量

http://www.example.com/test?id=&sort=ascending →
http://www.example.com/test

- 10.删除多余的查询串, 如?,&

http://www.example.com/test? →
http://www.example.com/test

- 11.[Dust 规则](#)(Schonfeld 等人提出的启发式方法)

http://www.example.com/test?id=123 ->
http://www.example.com/test_123